DATA MANAGEMENT I (DTS304)

Course Information & Introduction

Aims

To introduce **key concepts** and **practical skills** in database design and implementation

To explore the main features of a DBMS, key data management concepts and standards

To introduce key concepts database security and management databases.

Learning Outcomes

Identify, handle and manipulate **structured data** using modern databases

Efficiently handle and manipulate **large datasets**

Identify and implement appropriate **data management techniques**

Apply **analysis techniques** to extract knowledge from data

Outcomes in Simpler Words

what you will be able to do?

- 10 weeks on relational databases
  - Will use MySQL as the DBMS

Lecturer

Abdullahi Ahamad Shehu

email: aashehu.cs@buk.edu.ng

Email to ask questions/arrange for meeting

Delivery of the Course

pre-recorded lecture video will share in the class group

- **watch before you come to the class or labs**

Assessment

1 coursework (30 Marks)

- Design a relational DB

deadlines:

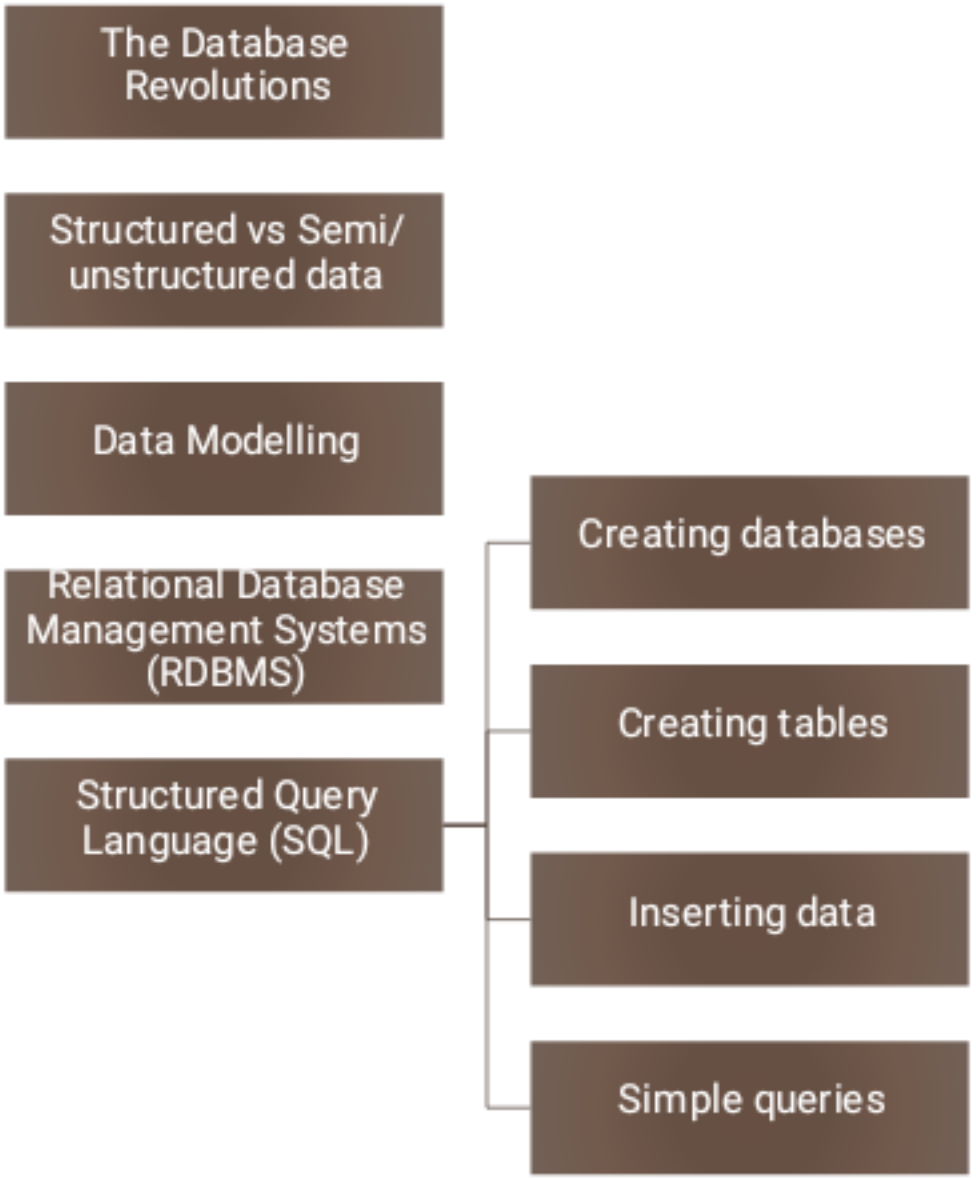
- **13/07/2026, 16:00 (GMT +1)**
- 2 Exams (70 Marks)

Books

CONNOLLY,T and BEGG,C., 2015. Database Systems: A Practical Approach to Design, Implementation and Management. Pearsons.

ELMASRI, R and NAVATHE, S.,2011. Fundamentals of Database Systems. Addison Wesley

Content


The Database Revolutions
Structured vs Semi/ unstructured data
Data Modelling
Relational Database Management Systems (RDBMS)
Structured Query Language (SQL)
Creating databases
Creating tables
Inserting data
Simple queries

Definition

Data Management is the practice of collecting, organizing, protecting, and storing data to analyze and use it effectively for decision-making.

Key Components:

Data Governance: Policies, rules, and standards for data usage.

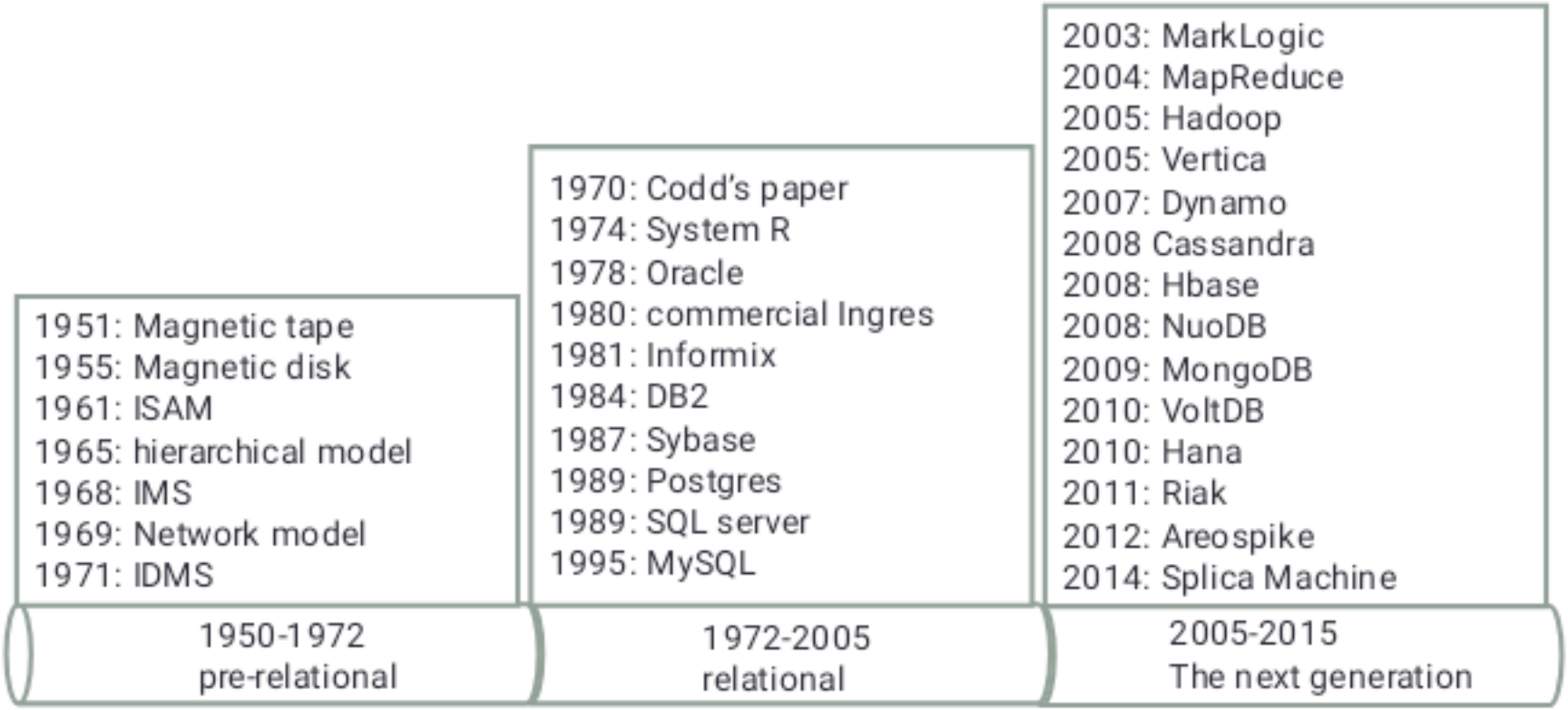
Data Quality: Ensuring accuracy, consistency, and completeness.

Data Lifecycle: From creation and storage to archiving or deletion.

Security & Privacy: Controlling access and complying with regulations (e.g., GDPR, HIPAA).

*Think of it as **the discipline** of treating data as a valuable resourcesimilar to managing finances or inventory.*

The 3 Database Revolutions



- Figure from G. Harrison, Next Generation Databases. Apress. 2015.

The Changing Needs

- 1st transition
  - Start storing data in computers
- 2nd transition
  - relational databases make data more available to non-programmers
- 3rd transition
  - the bloom of the Internet means data volume has increased enormously
  - recognising the value of data (when you have a lot)
  - data structure is more loose

Structured vs Semi/Un-structured Data

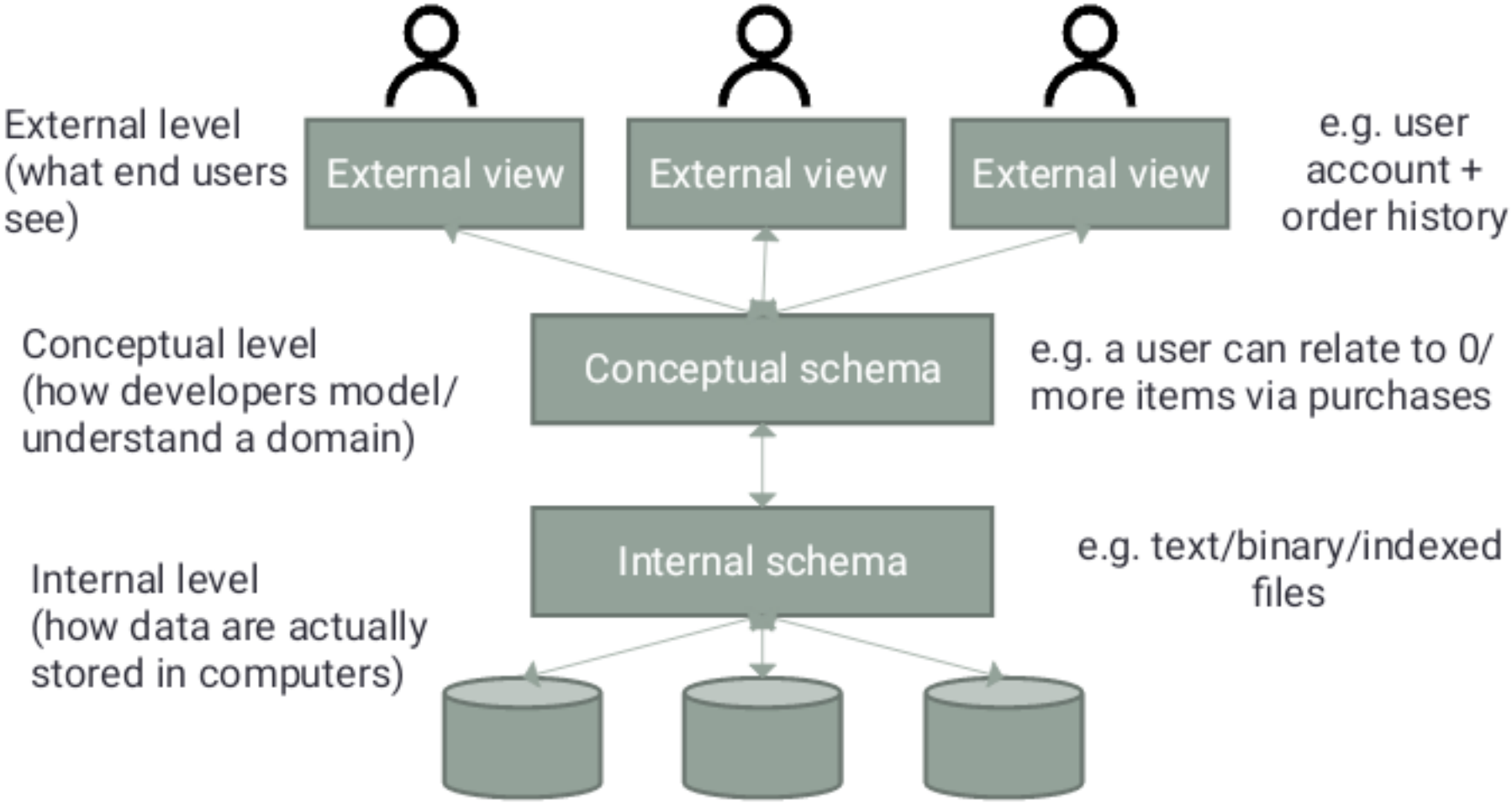
- Structured data
  - All instances follow the same (domain-dependent) structure
  - E.g. each student has a name, matriculation no., address
  - Each student studies multiple modules in a semester
  - Different instances may have different values
  - E.g. student/employee records, products in an online shop, customer orders
  - students may have different names, but they all have a name, a student number, DOB, etc.
- semi/un-structured data
  - Not much structure
  - OR structure may change in different instances
  - E.g. web server access log, data from sensors, optional data fields

Managing Structured Data

Traditional databases require data to follow a pre-defined structure

- structure must be defined before data are stored
- called the **database schema**

Database Systems – 3-level Architecture

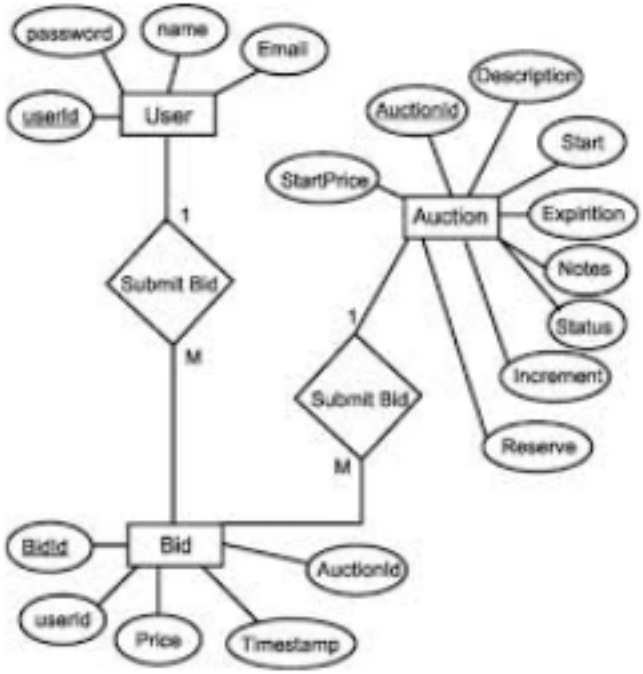


3-level Architecture (cont'd)

- **External** (user logical, view) level
  - Closest to users' understanding of the domain
  - Can hide parts of the database while exposing some
  - E.g. the list of items/bids you see in eBay

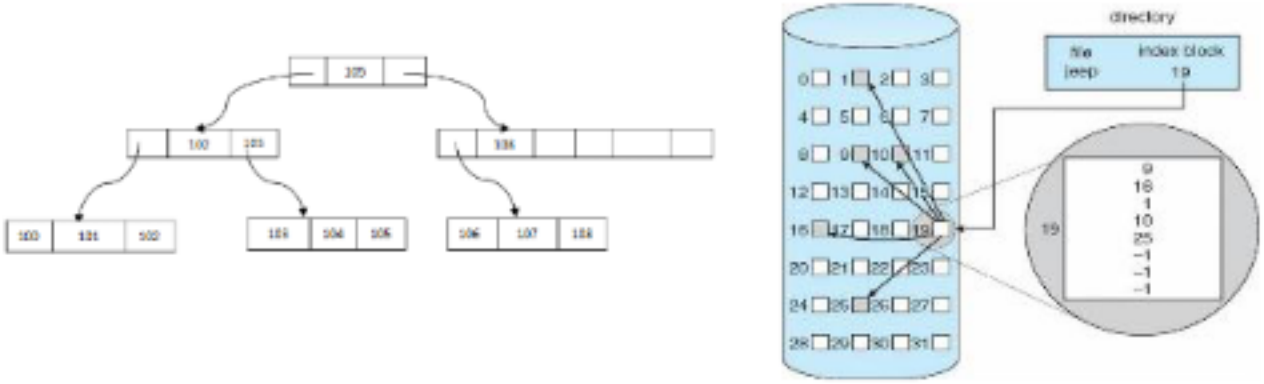
3-level Architecture (cont'd)

- **Conceptual** (logical) level
  - Hides details of physical level
  - Focus on describing entities, relationships in domain
  - E.g. an item may have 0 or multiple bids in eBay
- each seller may have 0 or more items for sale



3-level Architecture (cont'd)

- **Internal** (physical) level
  - Closest to physical storage, how data are physically stored
  - E.g. trees/ indexed files on hard disks



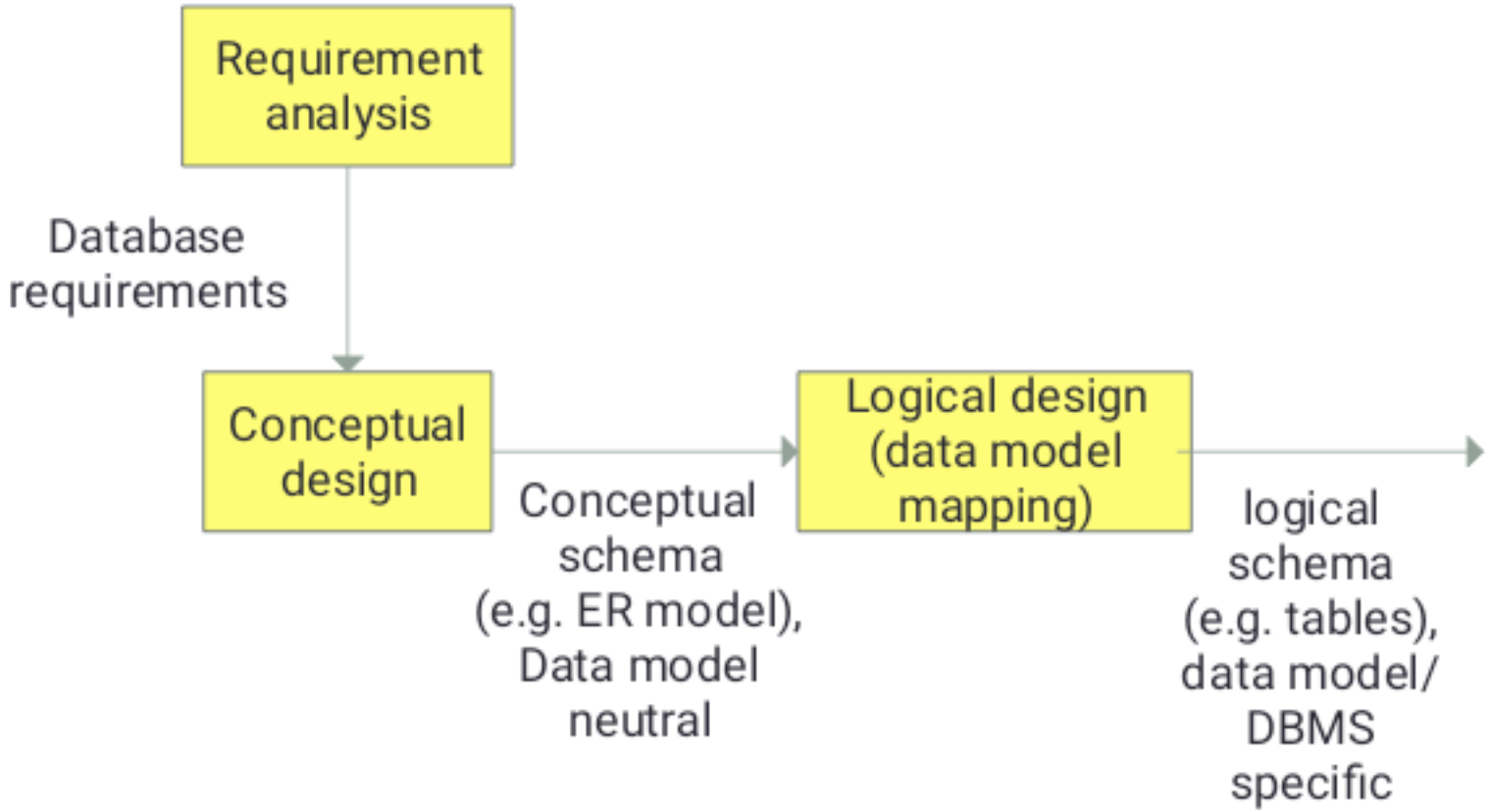
Data Modelling

- Data model: a collection of concepts to describe the structure of a database
- To manage data, we start by recognising "**things of interest**" in a domain
  - relevant information are captured
  - irrelevant details are discarded
- Build a **conceptual schema**
  - Note: no need to worry about the internal schema as it is taken care of by the DBMS

Database, Schema & Data Instances

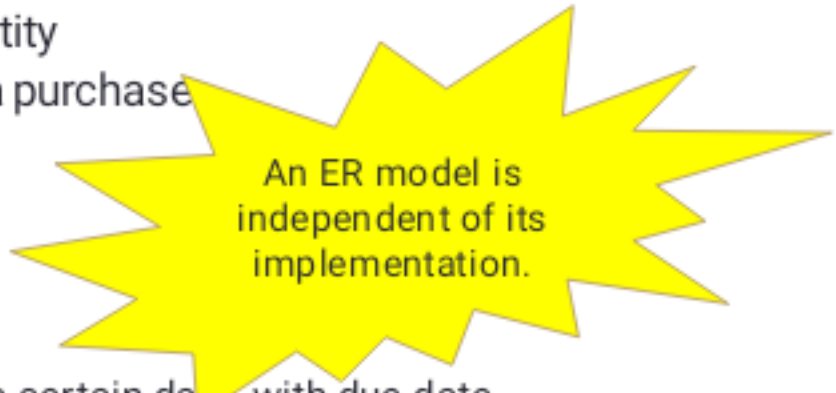
- A **schema** describes the structure of a database
  - must be designed before database can be created
  - **A schema does not change frequently**
  - E.g. a student has a name which is a string, and a student number which is an integer, a student takes 1 or more modules
- **Data instances** are actual data stored in the database
  - **Data may change frequently**
- Data in a database at a particular moment of time is called a **database state** or **snapshot**

Major Phases in Database Creation



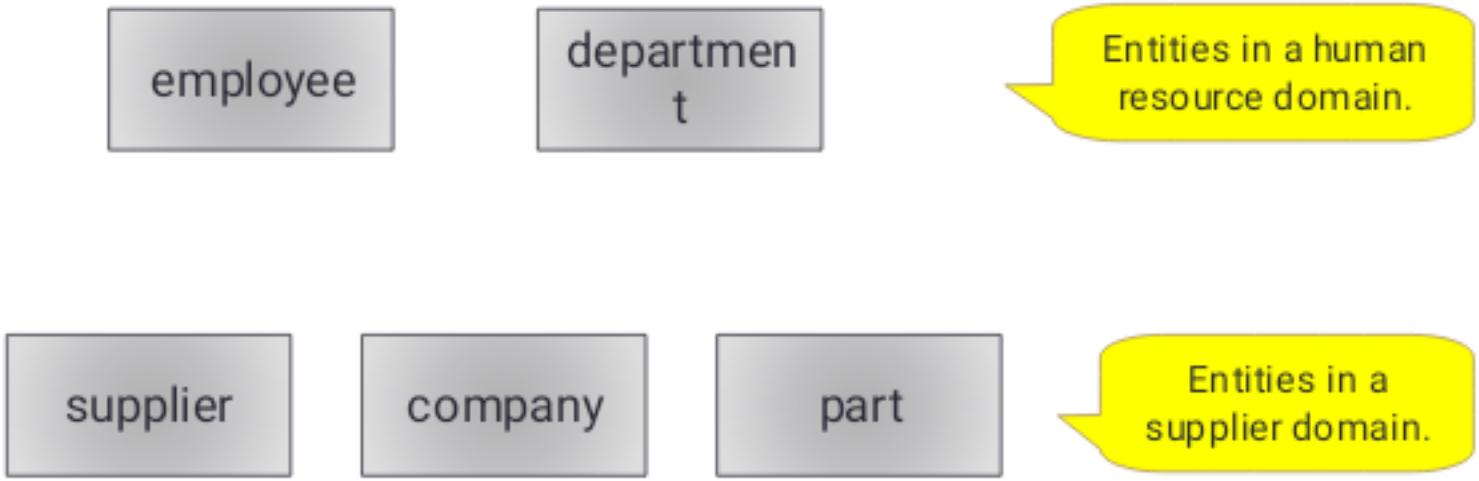
Conceptual Data Modelling

- **Entity-Relationship modelling** is a conceptual modelling method
- Analyse data requirements to identify:
  - **Entities**
  - A distinguishable object in the domain
  - E.g. an employee, a student, a car, an order/purchase
  - **Attributes**/properties
  - A piece of information that describes an entity
  - E.g. the name of an employee, the date of a purchase
  - **Relationships**
  - Connects 2 or more entities
    - E.g. a customer ordered an item
  - May have properties
    - E.g. a book is borrowed by a library user on a certain date, with due date
  - **Constraints**/restrictions



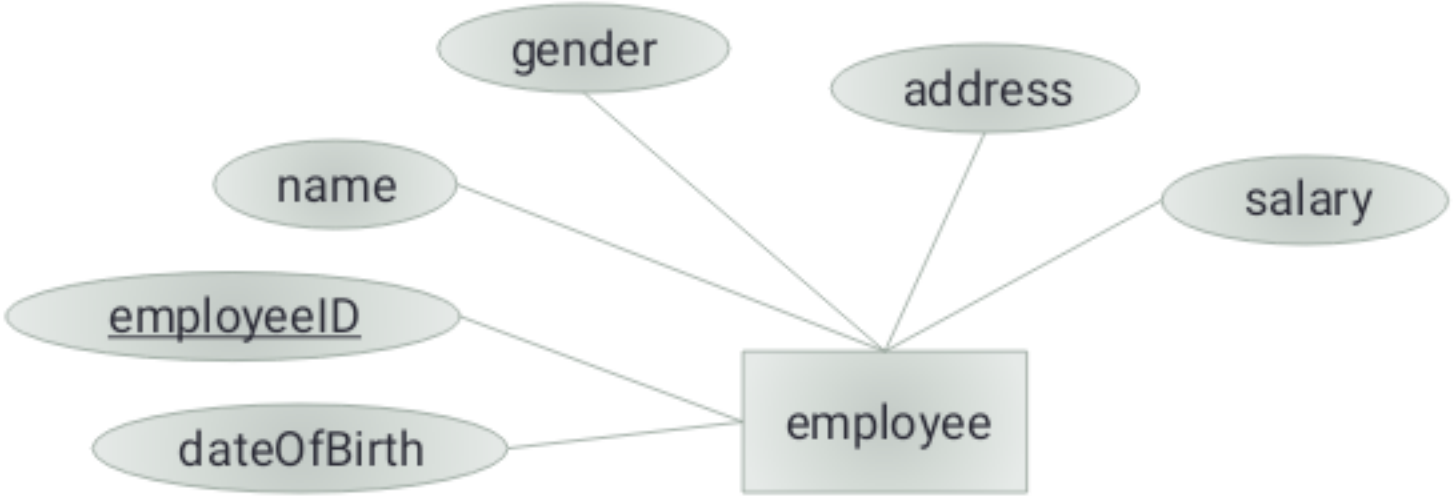
Entity-Relationship Modelling – Entities

- Basic objects in an ER model
- Models a physical object or conceptual existence
- **You need to identify objects of interest that you want to keep/store**



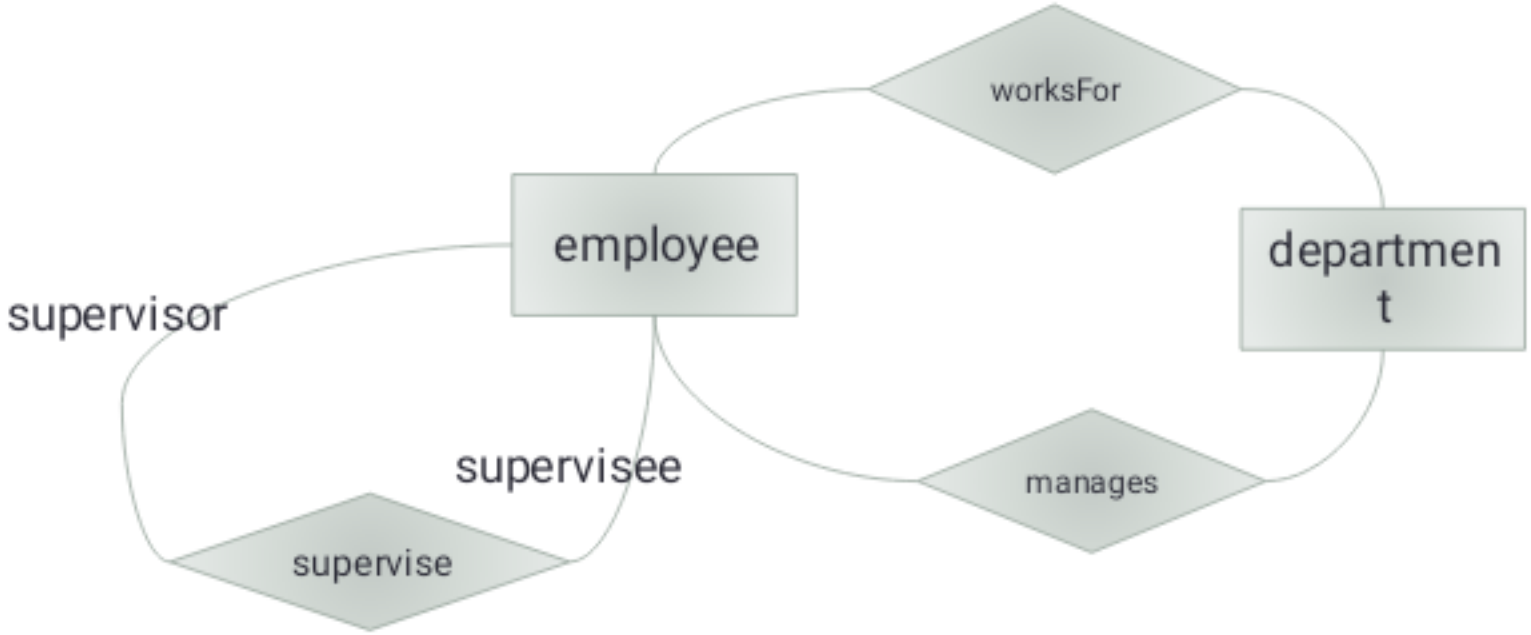
ER Modelling – Attributes

- An attribute describes an entity or relationship
- A key attribute uniquely identifies an instance of the entity/relationship
  - Key attribute is underlined
- **Your need to identify properties of interest on entities**



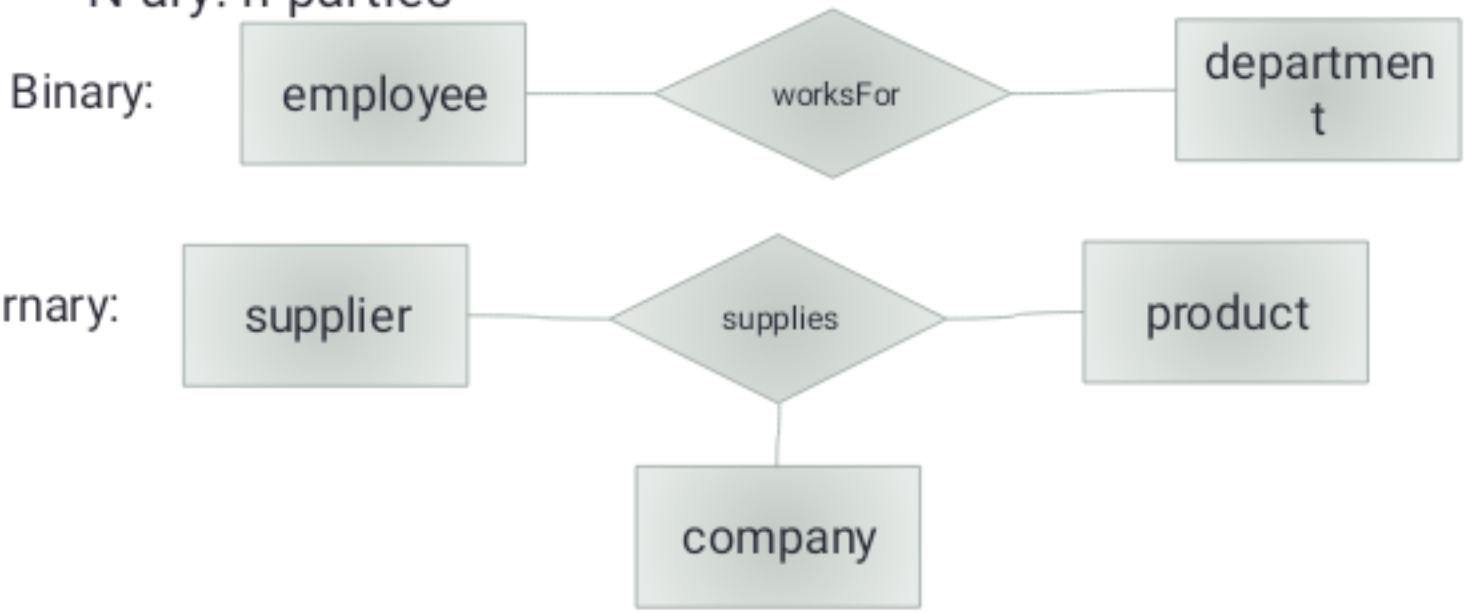
ER Modelling - Relationship

- A relationship is an association among entities
  - May relate 2 or more entities
- **You need to identify relationships of interest linking entities**



Degree of Relationship

- The number of entity types participating in the relationship
  - Binary: 2 parties
  - Ternary: 3 parties
  - N-ary: n parties



1-to-1 Relationship

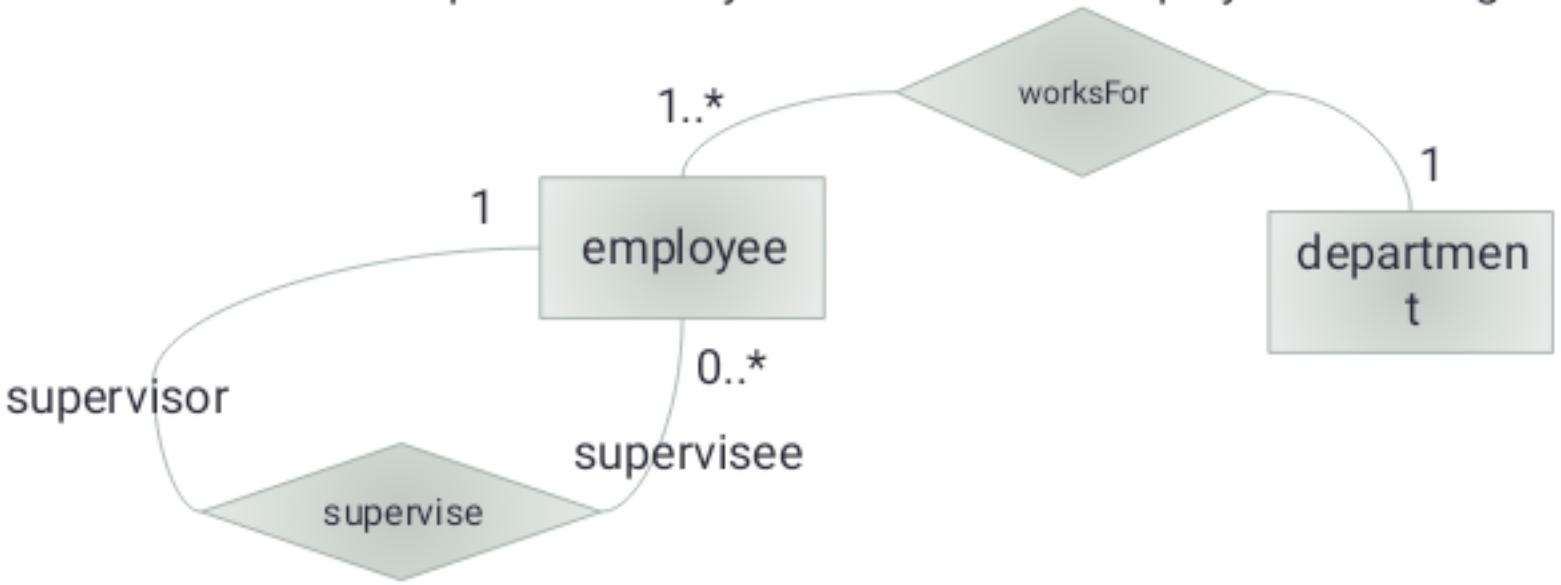
- 1 instance of an entity relates to exactly 1 instance of another entity

An employee manages 0 or 1 department
A department is managed by exactly 1 employee.

1-to-Many Relationship

- 1 instance of an entity can relate to multiple instances of another instance

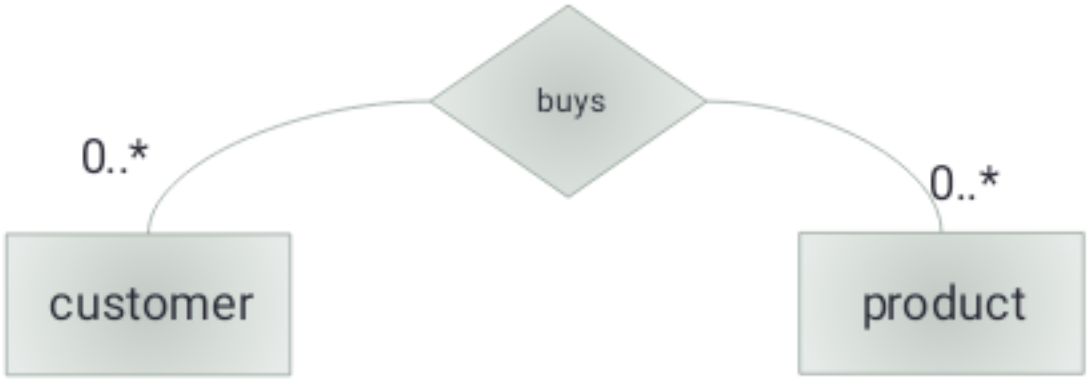
An employee works for exactly 1 department.
A department may have 1 or more employees working for it.



An employee supervises 0 or more (other) employee(s).
An employee is supervised by exactly 1 (other) employee.

Many-to-many Relationship

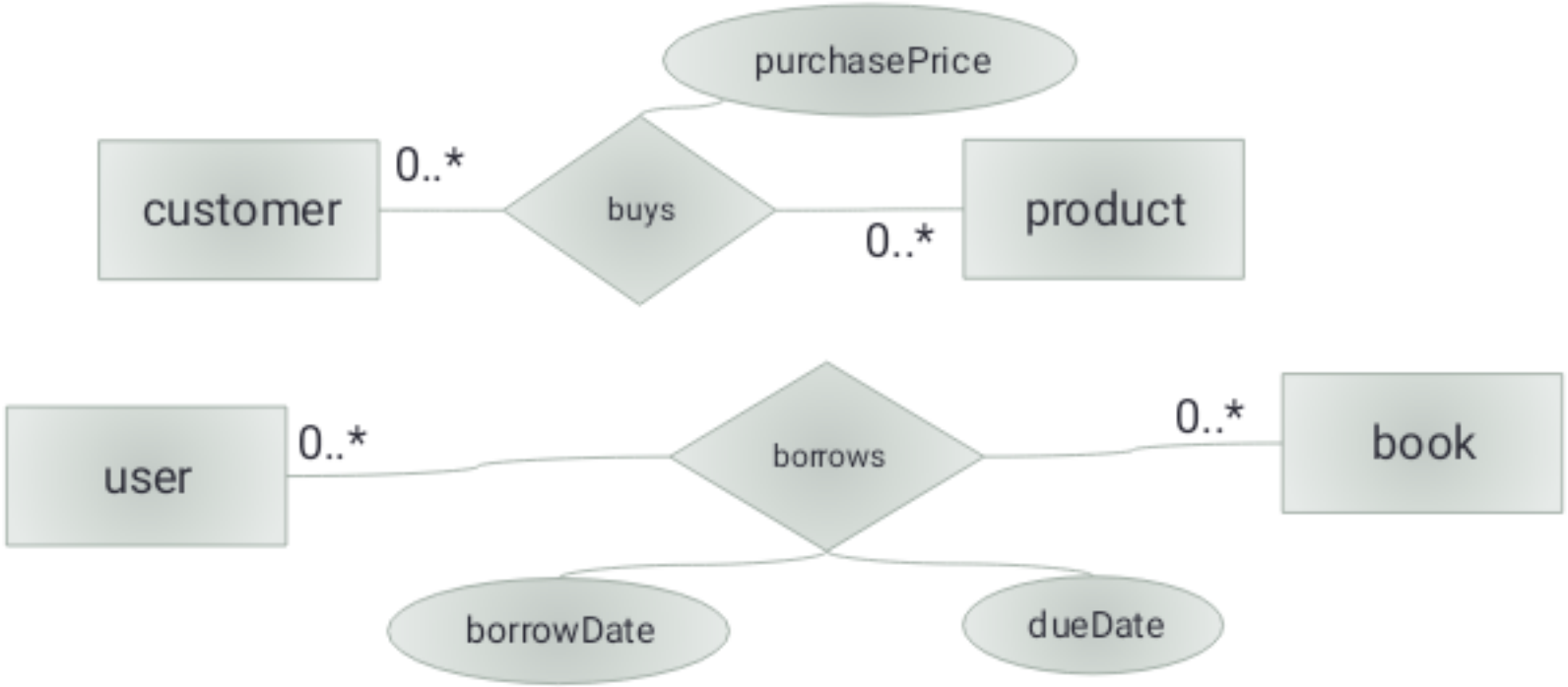
- Many instances of an entity can relate to many instances of another entity



A customer buys 0 or many products.
A product is bought by 0 or many customers.

Relationship with Attribute

- **The attribute value is attached to an instance of the relationship**
  - E.g. even the same customer buying the same product again may have a different purchase price



The Conceptual Model

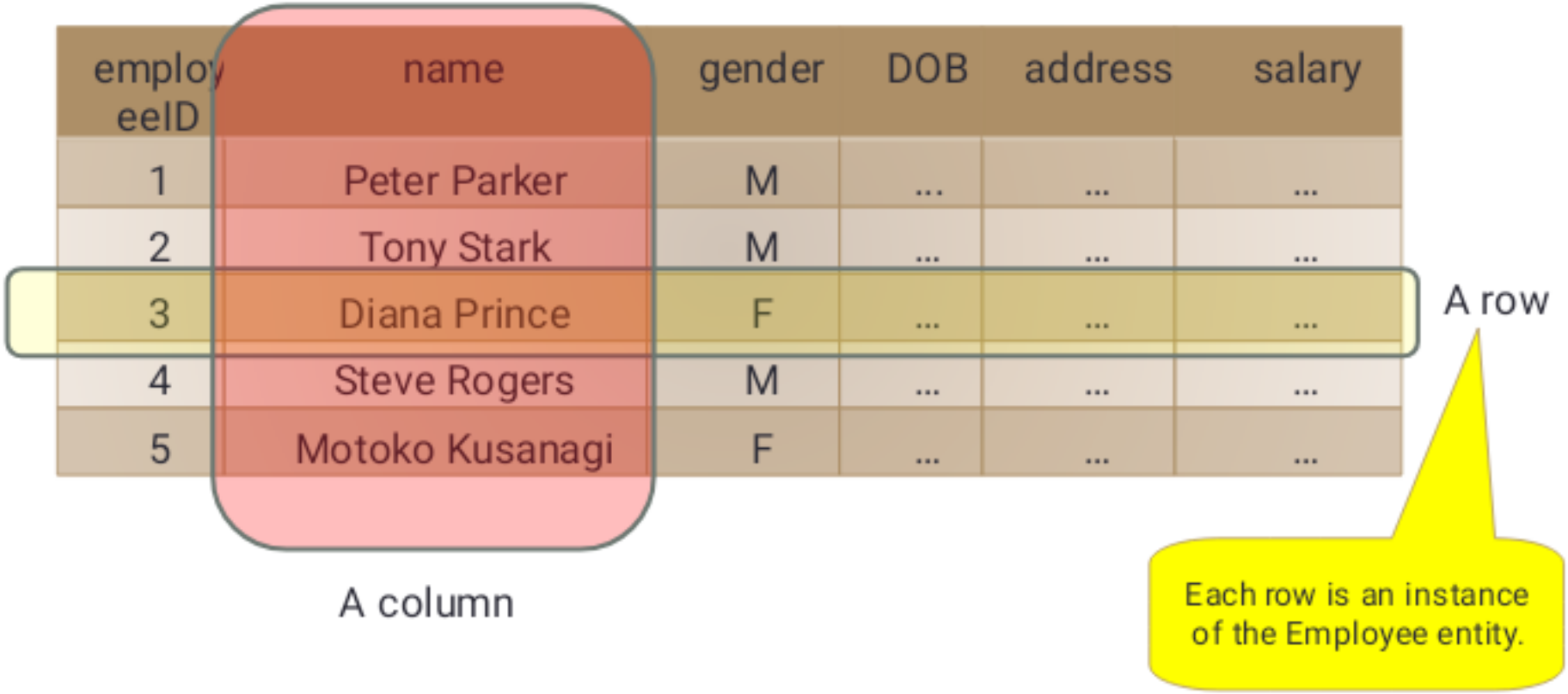
- an ER model is only a conceptual model
  - it helps you to understand the data
  - **it DOES NOT tell you how to implement it**
- To implement a given conceptual model as a database, you need to choose a data model
  - E.g. the **relational data model** uses relations to implement a conceptual model
  - the **functional data model** uses functions
  - The **object data model** uses objects
  - The **hierarchy data model** uses hierarchies (i.e. trees)
- then map your conceptual model into a **schema** (specific to a data model)

We will look at the RDM as it is the most common one.

The Relational Data Model (RDM)

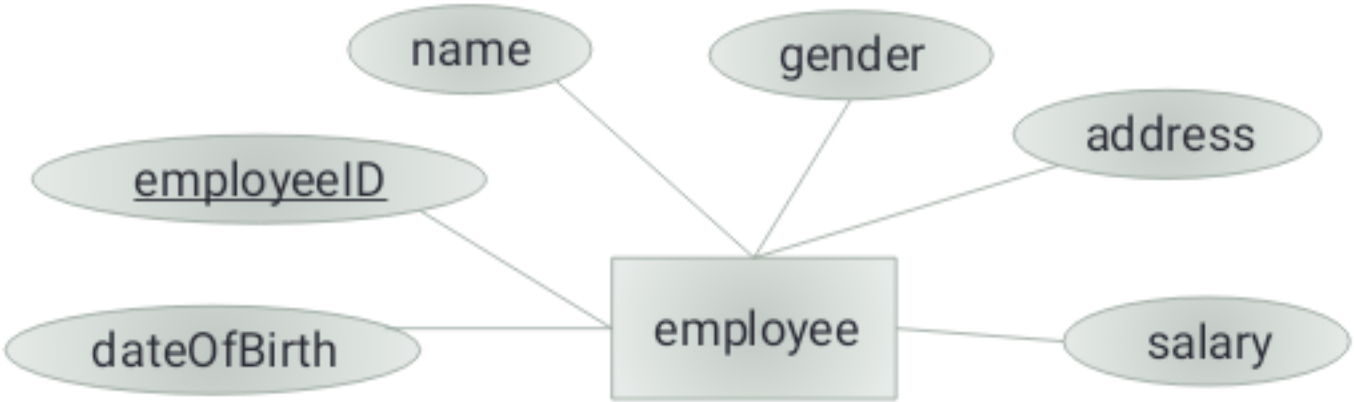
- Model data as **relations**
  - Don't confuse with *relationships* in ER modelling
  - Can be used to implement an ER model
- A **database** contains **tables** (relations)
- A table has **columns**/fields (i.e. attributes)
  - Defined by database schema
- A **row** (i.e. record) is an instance of an entity
- in RDM, a **database schema** is a set of table structures

Tables, Columns & Rows



ER to Relation Mapping - Entity

- For each entity in the ER model:
  - defines a table
  - Each attribute becomes a column in the table
  - Key attribute becomes key of the table



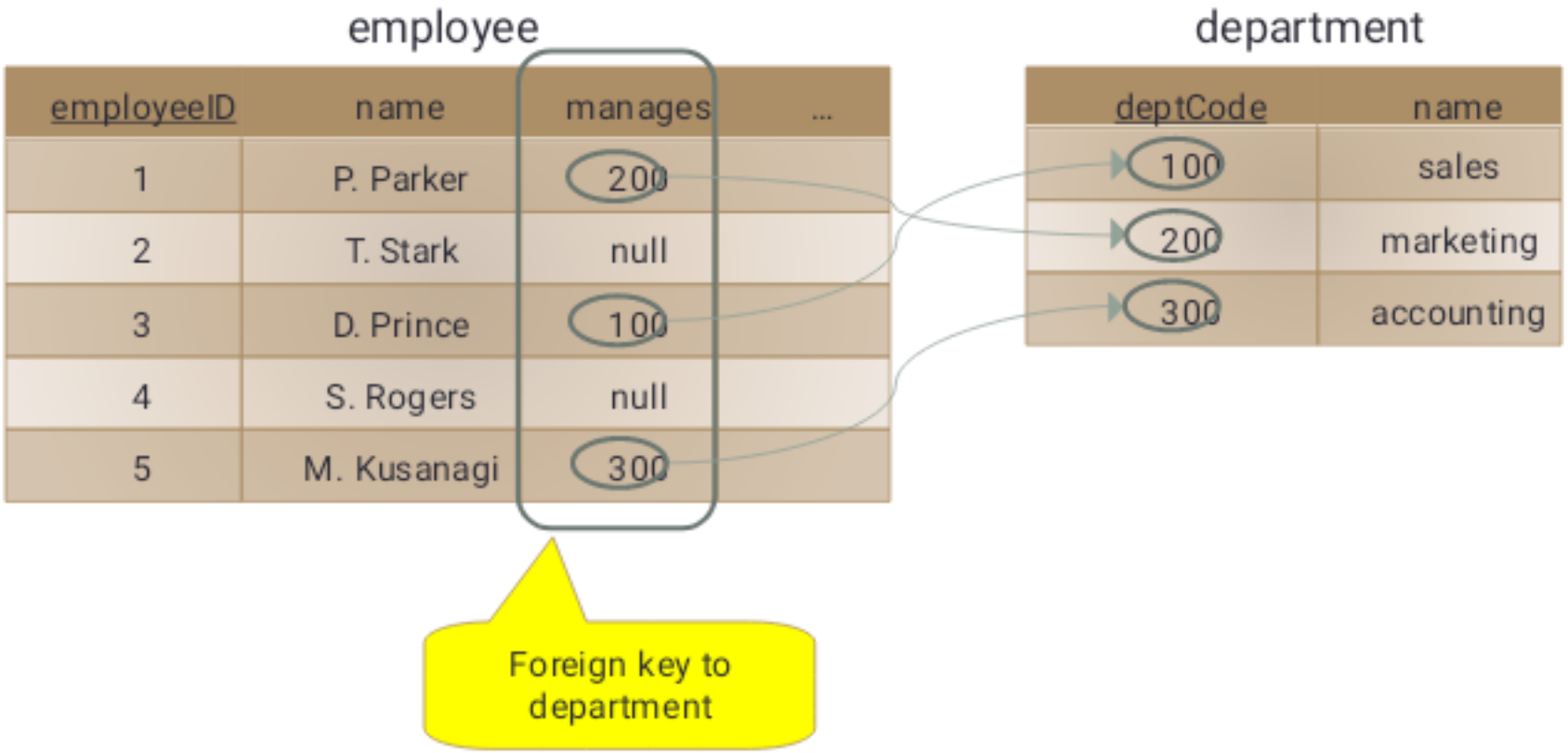
Entity to Table Mapping Example

employeeID	name	gender	DOB	address	salary
1	Peter Parker	M	...	...	...
2	Tony Stark	M	...	...	...
3	Diana Prince	F	...	...	...
4	Steve Rogers	M	...	...	...
5	Motoko Kusanagi	F	...	...	...

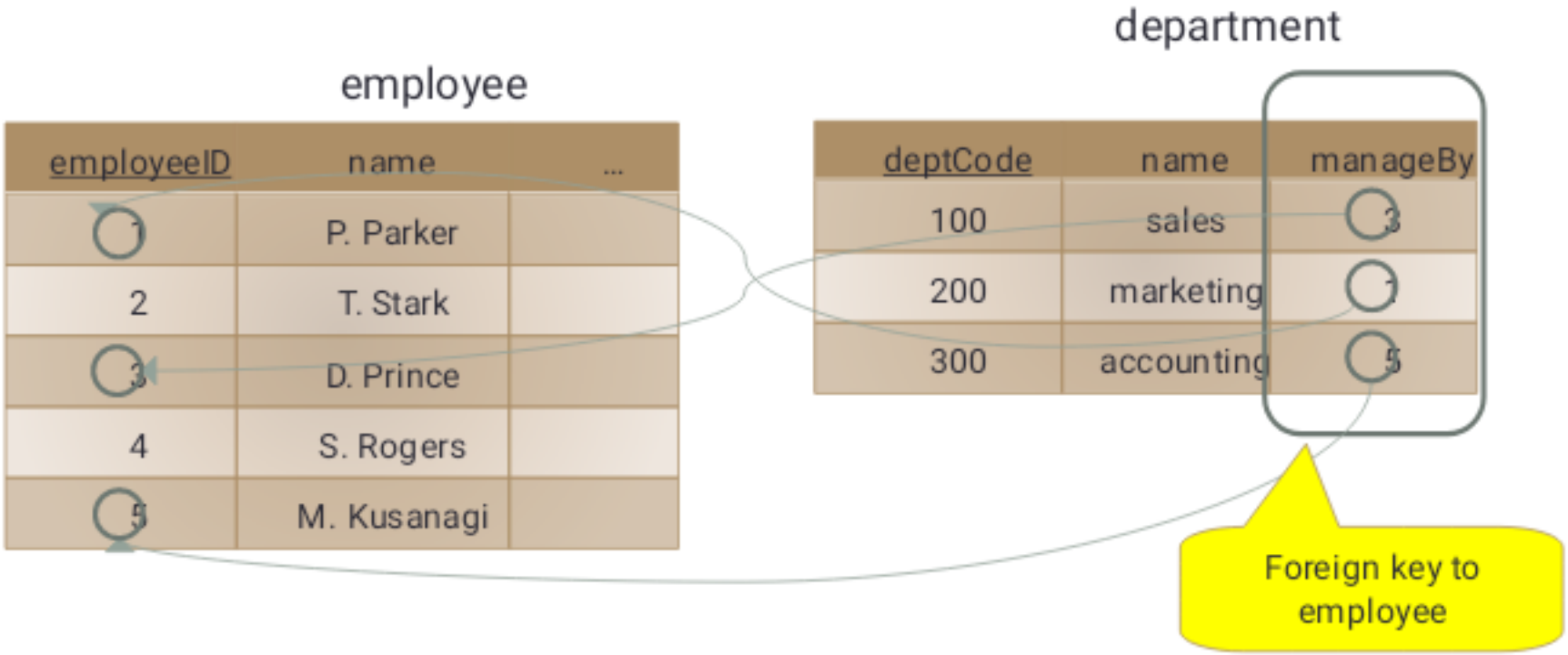
ER to Relation Mapping – Binary 1-to-1 Relationship

- For a 1-to-1 relationship between entity1 and entity2 which are mapped into tables T1 and T2:
  - Add a foreign key of T1 into T2
  - OR a foreign key of T2 into T1

1-to-1 Relationship to Table Mapping Example



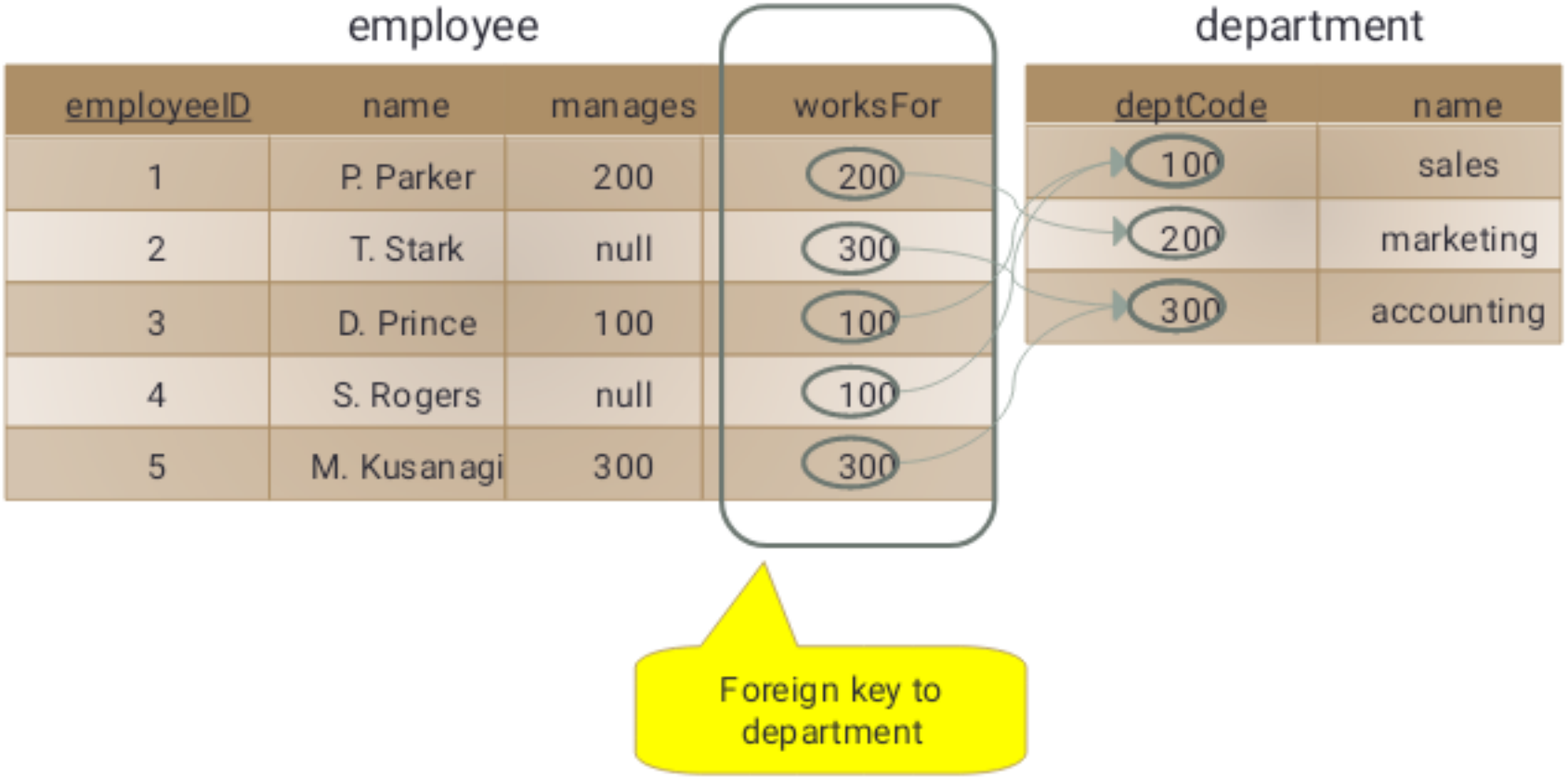
1-to-1 Relationship – Alternative Mapping



ER to Relation Mapping – Binary 1-to-many Relationship

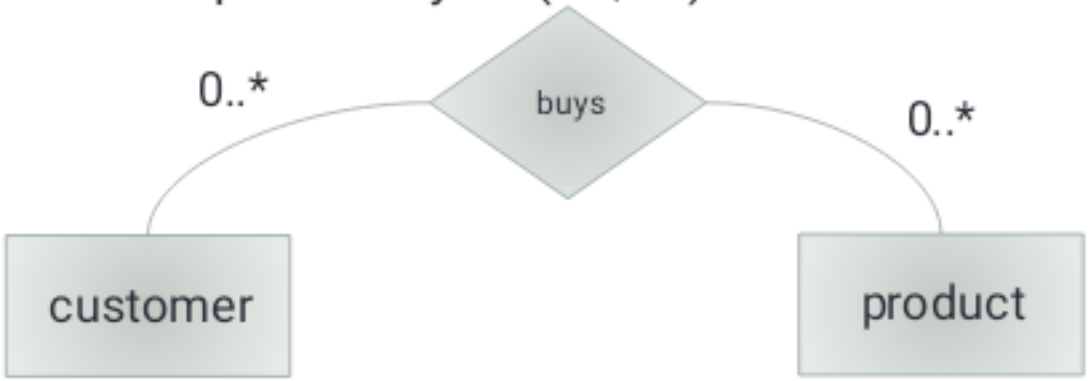
- For a 1-to-many relationship between entity1 & entity2 mapped into tables T1 & T2:
  - Add the foreign key of T1 into T2
  - i.e. foreign key goes to the "many" side of relationship

1-to-many Relationship Mapping Example

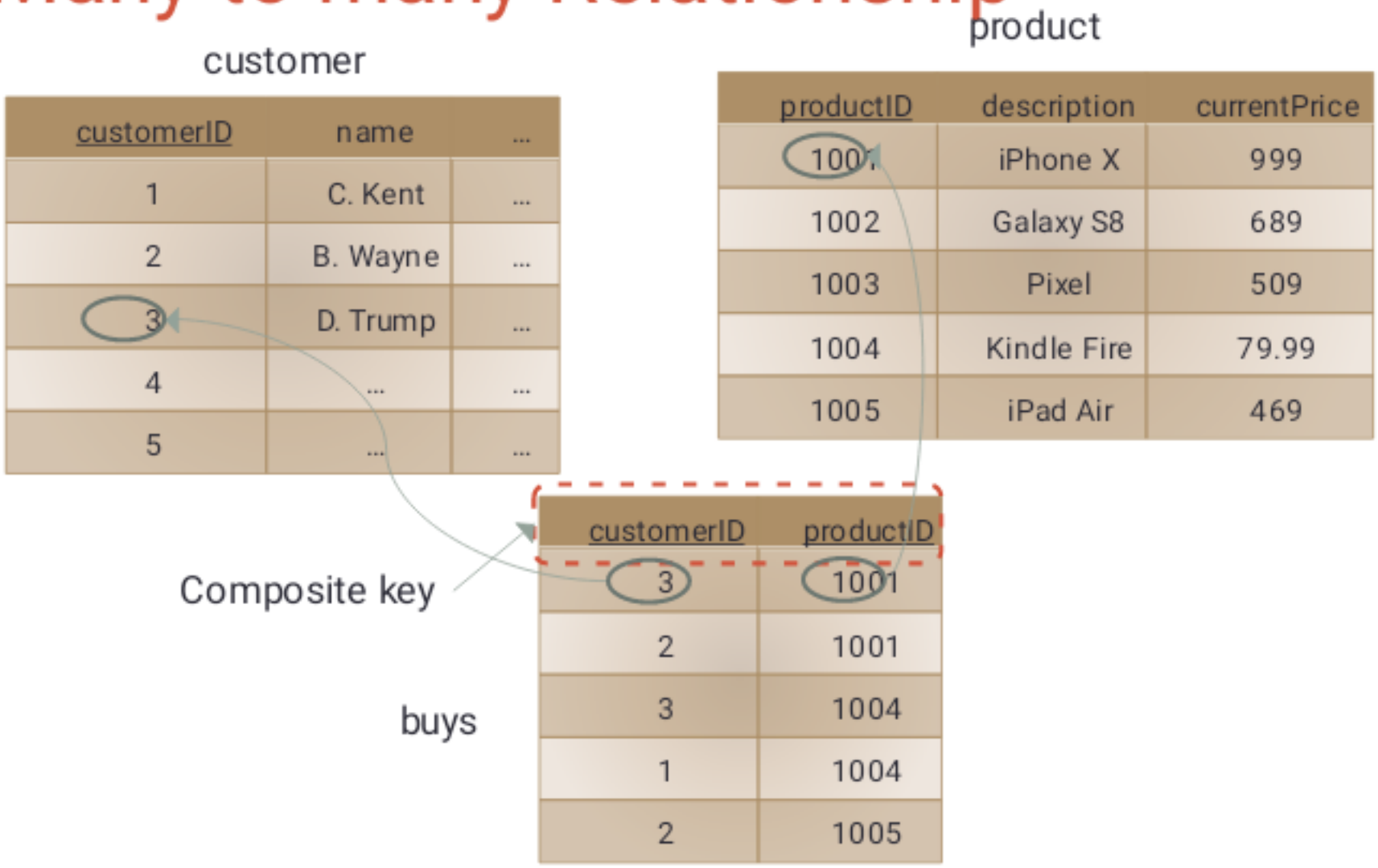


ER to Relation Mapping – Binary Many-to-many Relationship

- For a many-to-many relationship between entity1 & entity2
  - Assuming entity1 & entity2 are mapped in table T1 & T2, with keys K1 & K2 respectively
  - Create a new table R to implement the relationship
  - Add foreign keys of both T1 & T2 into R
  - R has the composite key of (K1,K2)

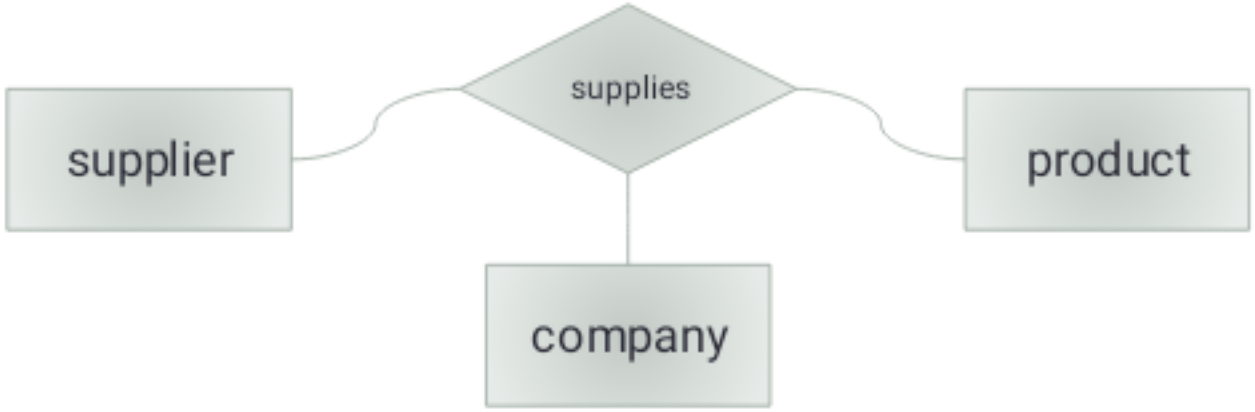


Many-to-many Relationship



ER to Relation Mapping – N-ary Relationship

- For a n-ary relationship among entity1, entity2, … entityn
  - Assuming entity1, …, entityn are mapped into tables T1, …, Tn with keys K1, …, Kn
  - Create a new table R
  - Include in R the foreign keys of T1, T2, …, Tn
  - Key of R is the composite key (K1, …, Kn)



N-ary Relationship to Table

company

companyID	name	...
C1	Microsoft	...
C2	Apple	...
C3	Google	...

supplier

supplierID	name	...
S1001	Amazon	...
S1002	Andrex	...
S1003	Samsung	...

Composite key

product

productID	description	...
PN500	Andrex rolls x6	...
PN600	Paper cups x100	...
PN700	Nescafe Gold 250g	...

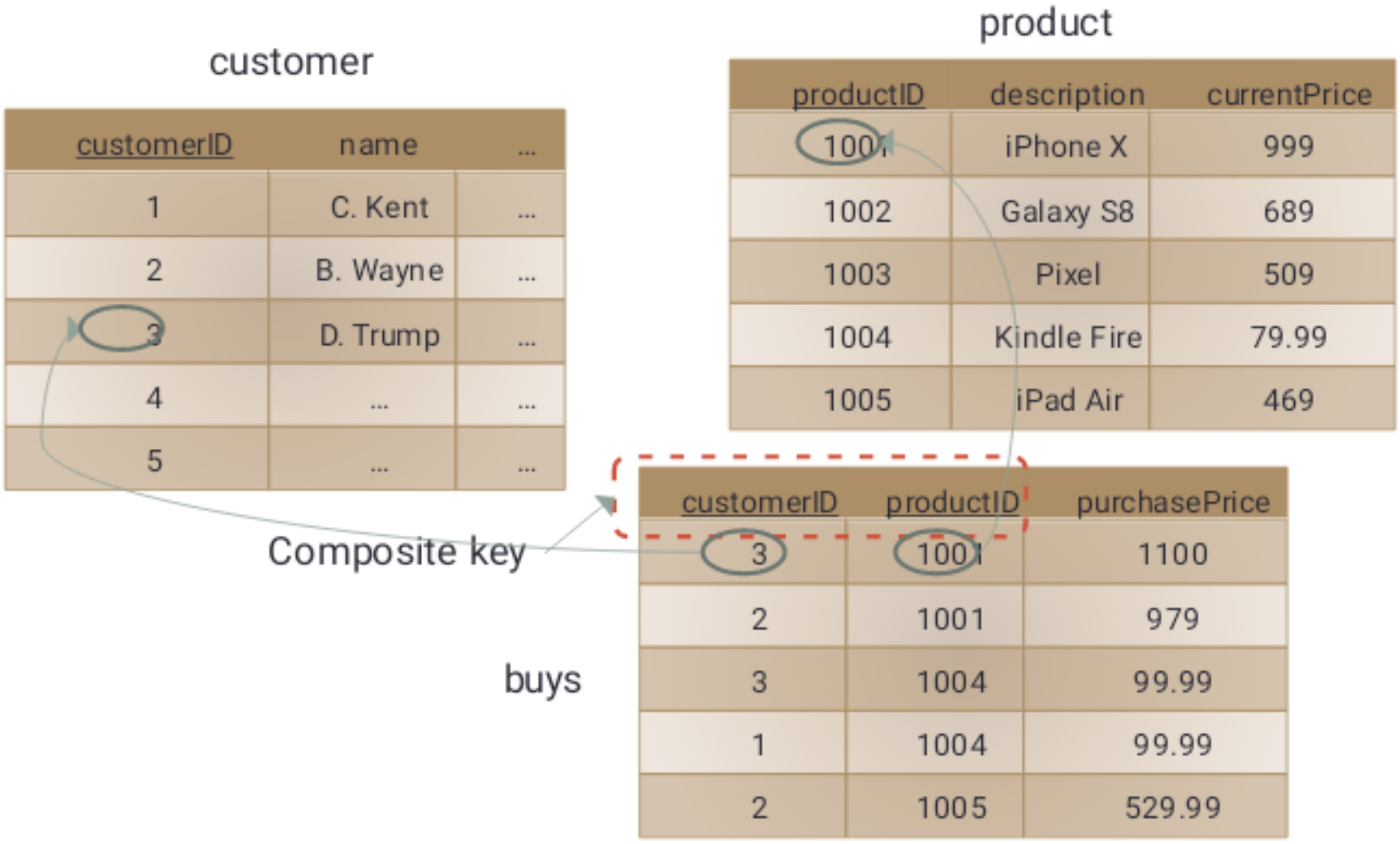
supplies

companyID	supplierID	partsNo
C3	S1001	PN600
C3	S1001	PN700
C2	S1001	PN700
C1	S1002	PN500
C3	S1002	PN500

Relationship with Attribute(s) Mapping

- For each relationship R with attribute(s):
  - Create an entity with attribute(s)
  - Then map entity to table
  - Table will have foreign keys of all entities involved in R

Relationship with Attribute to Table



Database Normalisation

The process of efficiently organising data in a database:

- Avoid redundancy (which may lead to inconsistency)
- Improve data integrity

Result in a good database structure

To be discussed later